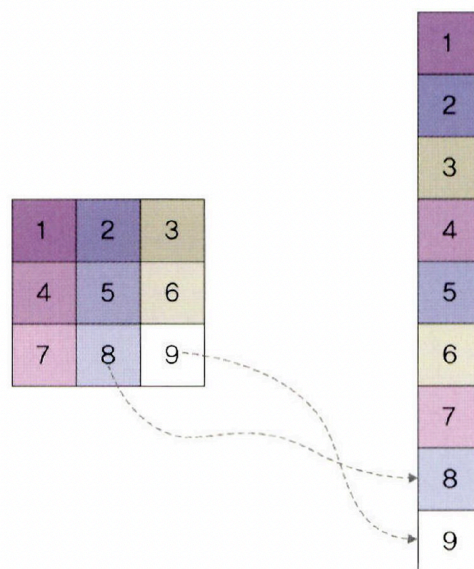


3주차

5.1 합성곱 신경망

5.1.1 합성곱층의 필요성

- 완전연결 신경망은 이미지를 그대로 펼쳐서(flatten) 입력으로 사용



- 문제점:
 - 이미지의 공간적 구조(픽셀 간의 위치 관계)가 무시됨.
 - 계산량이 매우 크고, 자원(CPU, GPU, 메모리) 소모가 큼.
- 해결책: 합성곱 신경망(CNN)
 - 이미지를 전체로 계산하지 않고, 국소적인 부분(일부 영역)만 계산.
 - 연산량과 자원을 줄이면서도 이미지의 세밀한 특징을 잡아낼 수 있음.

5.1.2 합성곱 신경망 구조

1. 입력층 (Input layer)

- 원본 이미지가 들어옴 (예: 가로 × 세로 × 색상 채널).

2. 합성곱층 (Convolution layer)

- 필터(커널)를 사용해 이미지의 특징(엣지, 패턴 등)을 추출.

3. 풀링층 (Pooling layer)

- 크기를 줄이면서(압축) 중요한 정보만 남김. (예: Max Pooling)

4. 완전연결층 (Fully connected layer)

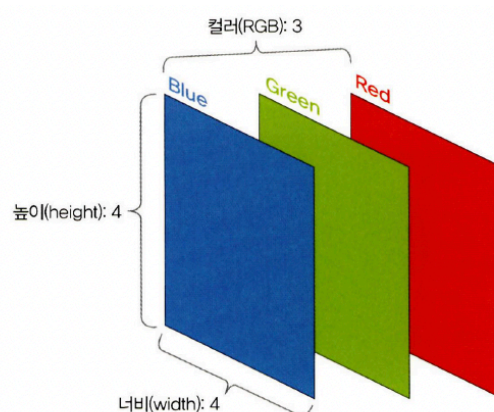
- 추출된 특징들을 1차원 벡터로 변환하여 학습/분류에 사용.

5. 출력층 (Output layer)

- 소프트맥스(Softmax) 등 활성화 함수를 통해 최종 결과 도출.
- 예: 강아지(0.01), 고양이(0.01), 보트(0.94), 자동차(0.94)

1. 입력층 (Input Layer)

- 입력층은 이미지 데이터가 처음 들어오는 층.
- 이미지는 3차원 데이터로 표현됨:
 - 높이(Height)
 - 너비(Width)
 - 채널(Channel)
- 예시:
 - 흑백(Grayscale) 이미지 → 채널 = 1
 - 컬러(RGB) 이미지 → 채널 = 3
- 따라서, 4×4 RGB 이미지는 (4, 4, 3) 형태를 가짐.



2. 합성곱층 (Convolution Layer)

- 합성곱층은 입력 데이터에서 특징(Feature)을 추출하는 역할.
- 커널(필터, Kernel/Filter) 이 이미지 위를 슬라이딩 하면서 영역별 연산 수행.
- 연산 결과는 특성 맵(Feature Map) 으로 출력됨.
- 합성곱 연산 방법
 1. 필터(예: 3×3)를 입력 이미지의 특정 영역에 겹침
 2. 대응되는 값끼리 곱한 후 모두 더함(가중합).
 3. 그 결과를 출력의 해당 위치에 기록.
 4. 필터를 Stride(이동 간격) 만큼 옮겨가며 반복
- 하이퍼파라미터 (Hyperparameters)
 - 합성곱층을 정의할 때 중요한 값들:
 - 필터 개수(K) → 추출할 특징의 개수, 출력의 깊이(Depth)가 됨.
 - 필터 크기(F) → 보통 3×3 , 5×5 등.
 - 스트라이드(S) → 필터가 이동하는 간격.
 - 패딩(P) → 경계 처리를 위해 입력 주변에 0을 채워 넣는 방식.
- 출력 데이터 형태
 - 입력 크기: $W_i \times H_i \times D_i$
 - 출력 크기 계산:
 - 너비: $(W_i - F + 2P) / S + 1$
 - 높이: $(H_i - F + 2P) / S + 1$
 - 깊이: K (필터 개수)
- 예:
 - 1단계:



○ 2 단계:



○ ...

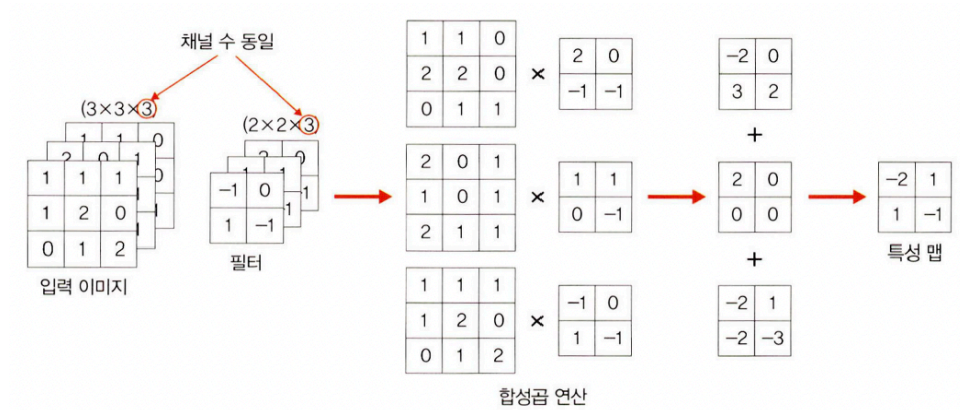
○ 마지막 단계



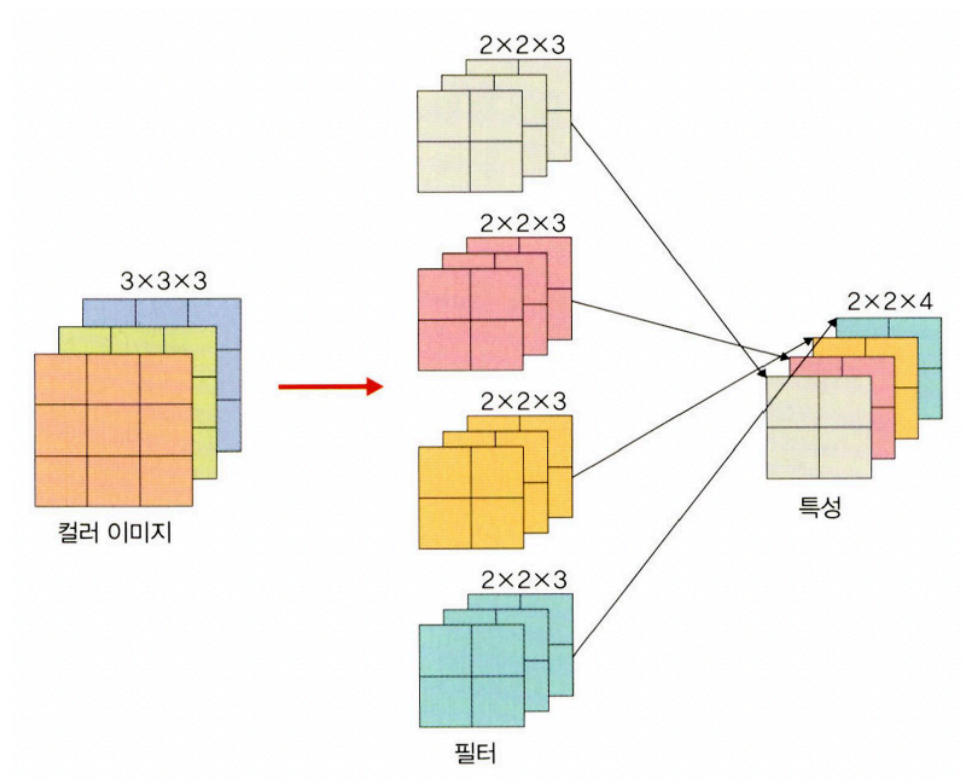
○ 계산 :

- 입력: $6 \times 6 \times 1$
- 필터: 3×3
- $W = 6 - 3 + 0 + 1 = 4$
- $H = 6 - 3 + 0 + 1 = 4$

- $D=1$
- 출력 크기 = (4, 4, 1)
- 컬러 이미지의 합성곱
 - RGB 이미지(채널=3)의 경우, 필터도 채널이 3개임.
 - 각 채널별 연산을 수행한 뒤, 그 결과를 모두 더해 하나의 특성 맵으로 만듦.
 - 예:

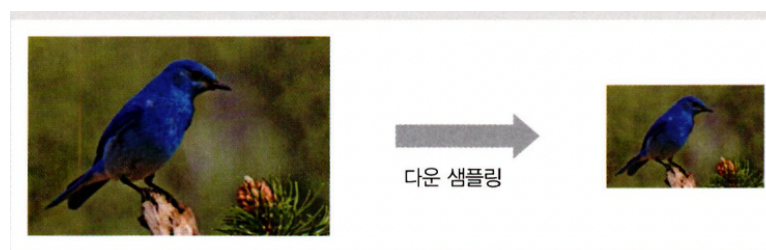


- 필터가 2 이상인 합성곱
 - 필터가 여러 개라면, 각 필터별로 특성 맵이 생성되어 출력의 Depth가 증가.
 - 예:



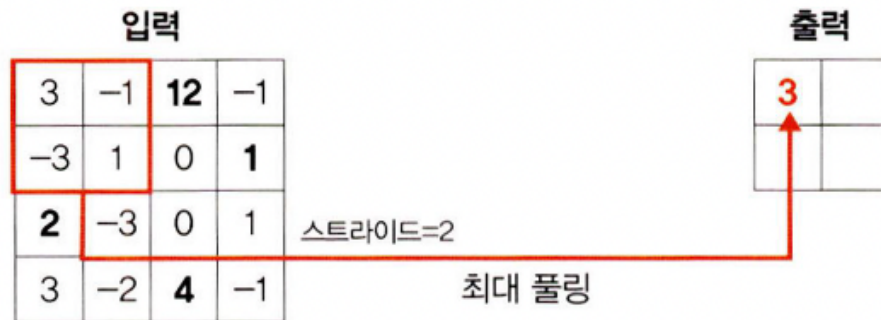
풀링층 (Pooling Layer)

- 역할: 합성곱층에서 얻은 특성 맵의 크기를 줄여 연산량을 감소시키고, 중요한 특징만 추출해 학습을 효율적으로 만듦.
- 다른 이름: 다운 샘플링 (Sub-sampling)

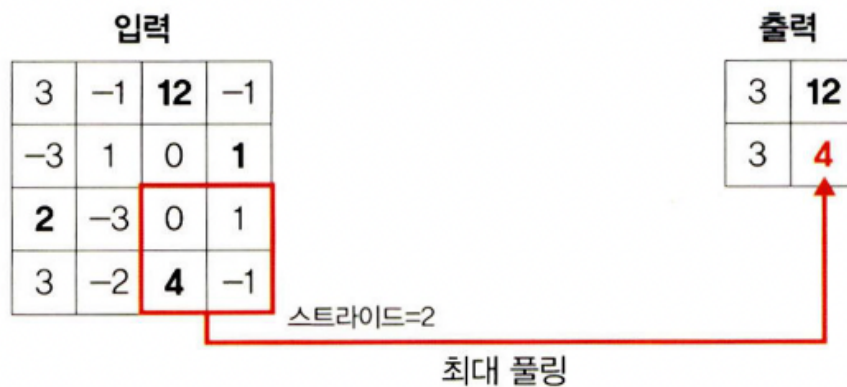


- 종류
 1. 최대 풀링 (Max Pooling)
 - 지정된 영역에서 최댓값을 추출.
 - 가장 많이 사용됨 → 중요한 특징을 잘 보존하기 때문.
 2. 평균 풀링 (Average Pooling)
 - 지정된 영역의 평균값을 계산.

- 하지만 강한 특징이 희미해질 수 있어 잘 쓰이지 않음.
- 최대 풀링 과정 예:
 - 1단계: 3, -1, -3, 1 중 최대값 선택



- 마지막 단계:



- 평균 풀링 : 계산법

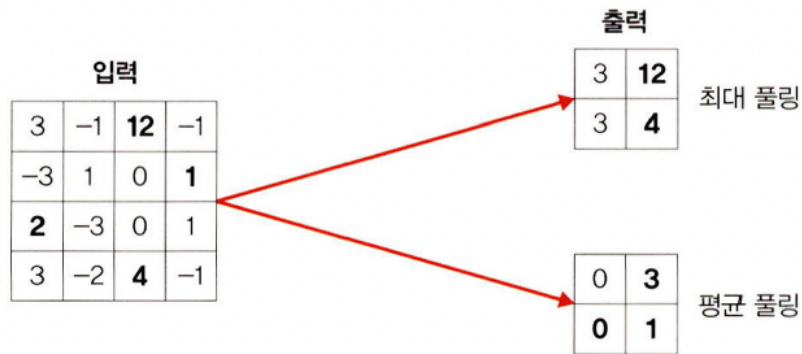
$$0 = (3 + (-1) + (-3) + 1) / 4$$

$$3 = (12 + (-1) + 0 + 1) / 4$$

$$0 = (2 + (-3) + 3 + (-2)) / 4$$

$$1 = (0 + 1 + 4 + (-1)) / 4$$

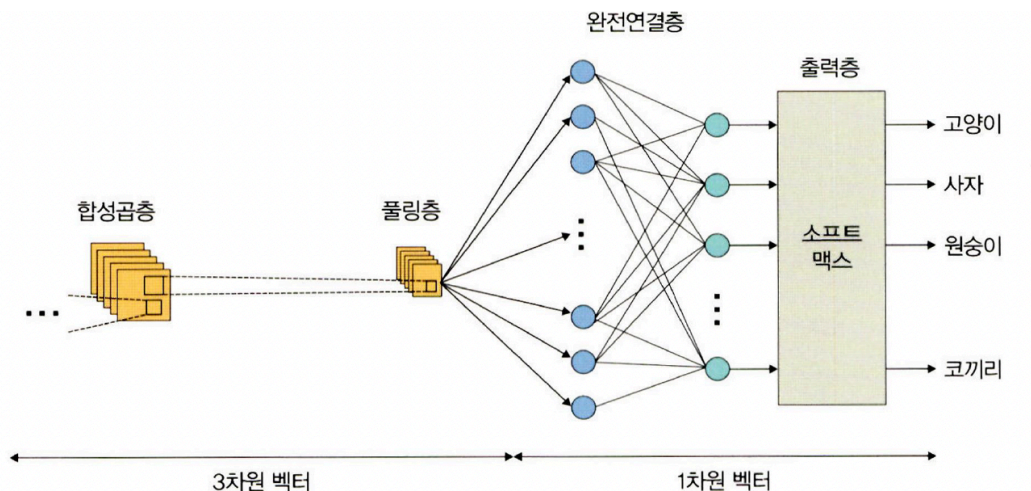
- 최대 풀링과 평균 풀링 비교



- 출력 데이터 형태 (Output Shape)
 - 입력 크기: $W \times H \times D$
 - 하이퍼파라미터: 필터 크기 F , 스트라이드 S
 - 출력 크기 계산:
 - 너비: $(W - F) / S + 1$
 - 높이: $(H - F) / S + 1$
 - 깊이: D

4. 완전연결층 (Fully Connected Layer, FC Layer)

- 합성곱층과 풀링층을 거친 뒤 → 특성 맵을 1차원 벡터(Flatten) 로 변환.
- 변환된 벡터가 완전연결층으로 전달됨.
- 완전연결층은 모든 뉴런이 서로 연결되어 있어, 추출된 특징을 종합하여 최종적인 분류 작업을 수행.



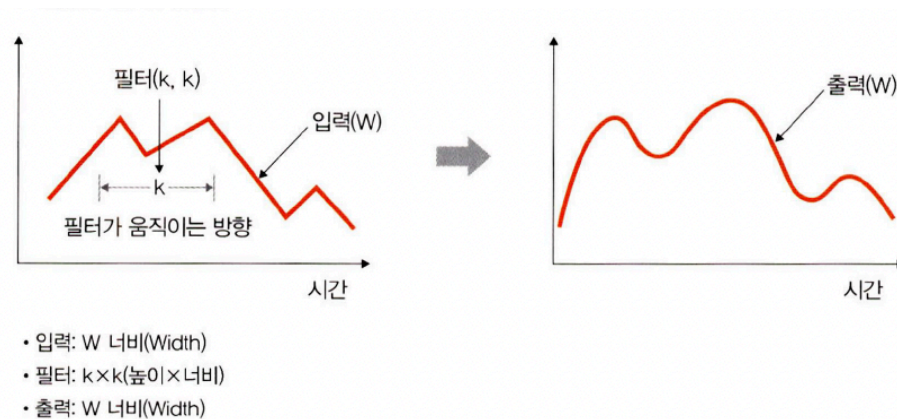
5. 출력층 (Output Layer)

- 주로 소프트맥스(Softmax) 함수를 사용.
- 소프트맥스 함수는 각 클래스에 대해 0~1 사이 확률값을 출력.
- 예시:
 - 고양이: 0.1
 - 사자: 0.05
 - 원숭이: 0.2
 - 코끼리: 0.65
- 가장 확률이 높은 클래스(코끼리:0.65) 가 최종 예측 결과가 됨

5.1.3 1D,2D,3D 합성곱

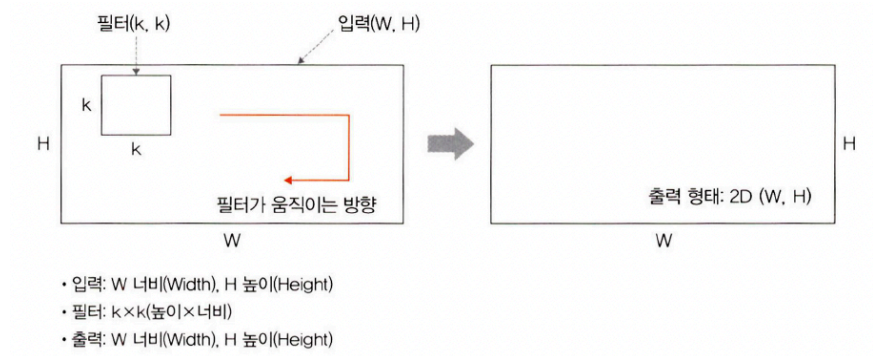
1. 1D 합성곱

- 필터가 한 방향(시간축 등)으로만 이동하는 합성곱.
- 입력(W)과 필터(k) → 출력은 1차원 배열(W).
- 주로 시계열 데이터, 음성 데이터 등에 사용.
- 예시: 입력 [1,1,1,1,1], 필터 [0.25, 0.5, 0.25] → 출력 [1,1,1].



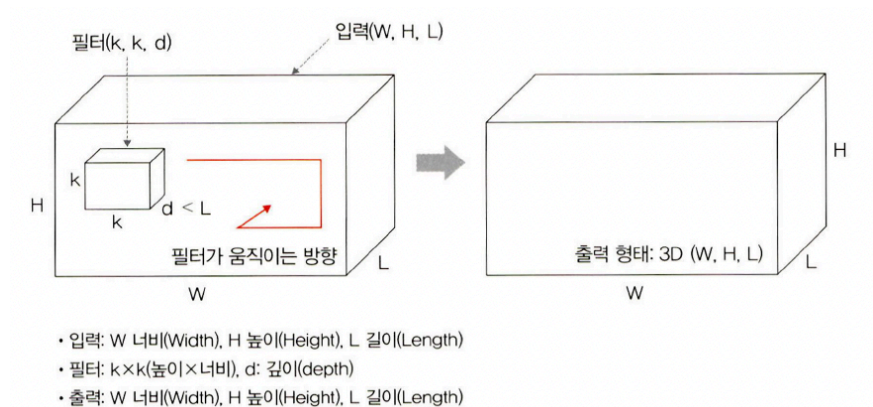
2. 2D 합성곱

- 필터가 가로(Width), 세로(Height) 두 방향으로 이동.
- 입력(W,H)와 필터(k,k) → 출력은 2차원 행렬(W,H).
- 주로 이미지 처리에 사용 (엣지, 패턴 추출).



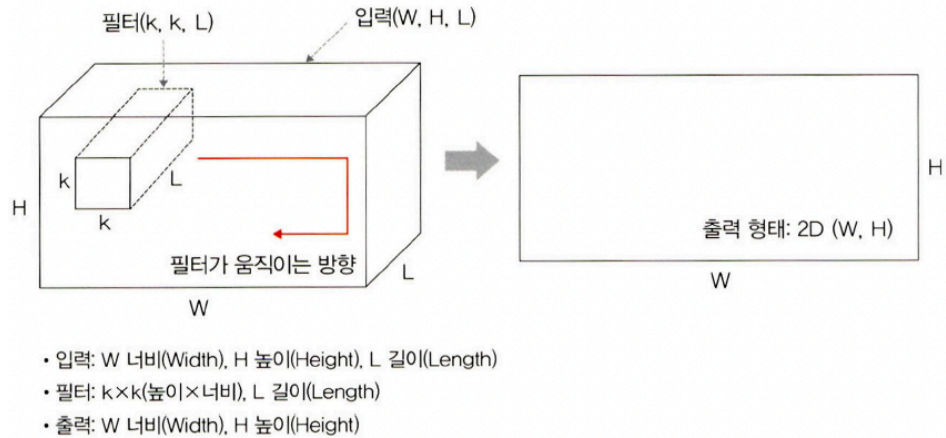
3. 3D 합성곱

- 필터가 가로, 세로, 깊이 세 방향으로 이동.
- 입력(W,H,L)와 필터(k,k,d) → 출력은 3차원 (W,H,L).
- 영상(동영상), 의료 영상(CT, MRI), 시공간 데이터 등에서 사용.
- 조건: 필터 깊이 d는 입력 깊이 L보다 작아야 함.



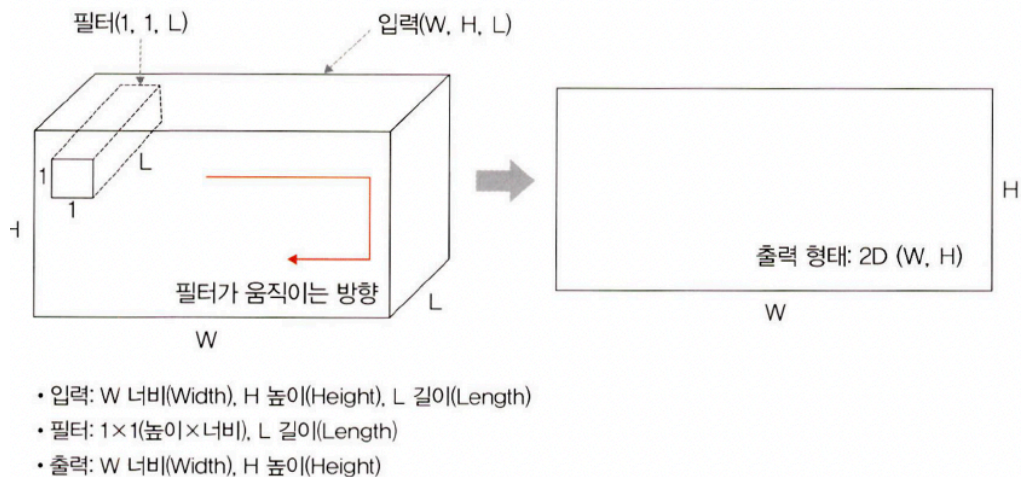
4. 3D 입력을 갖는 2D 합성곱

- 입력이 3D 형태라도(예: 컬러 이미지 224×224×3, 224×224×3), 출력이 2D 형태일 수 있음.
- 이유: 필터의 깊이가 입력 채널과 동일해야 하므로, 필터가 입력의 깊이를 모두 포함한 뒤 가로, 세로 방향으로만 이동.
- 출력은 2차원 (W,H).
- 대표 사례: LeNet-5, VGGNet.



5. 1×1 합성곱

- 입력이 3D 형태(W,H,L)일 때, 필터 크기를 (1,1,L)로 적용.
- 출력은 (W,H).
- 특징:
 - 각 픽셀 위치에서 채널 값들을 가중합 하여 새로운 채널 값 생성.
 - 채널 수를 줄이거나 늘릴 수 있음 → 연산량 감소 효과.
- 대표 사례: GoogLeNet (Inception Network).



5.2 합성곱 신경망 맛보기

Fashion-MNIST 데이터셋

- `torchvision` 에 포함된 예제 데이터셋.

- 작은 이미지(28×28 픽셀)로 구성, 총 70,000개 이미지.
- 기본 MNIST처럼 10개의 클래스(0~9)로 분류됨.
- 데이터 형식:
 - 훈련 데이터(train_images): 28×28 크기의 넘파이 배열, 값 범위 0~255.
 - 레이블(train_labels): 0~9 사이의 정수 (클래스 라벨).
- 클래스 종류:

0: T-shirt/top
 1: Trouser
 2: Pullover
 3: Dress
 4: Coat
 5: Sandal
 6: Shirt
 7: Sneaker
 8: Bag
 9: Ankle Boot

라이브러리 호출

```
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.autograd import Variable
import torch.nn.functional as F

import torchvision
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader
```

- numpy, matplotlib → 데이터 처리, 시각화.
- torch, torch.nn → 파이토치 핵심, 신경망 구성.
- Variable, F(functional) → 자동 미분, 신경망 연산.

- torchvision, transforms → 데이터셋 및 변환.
- DataLoader → 배치 단위로 데이터 로드.

CPU / GPU 설정

- 코드로 GPU 사용 가능 여부 확인:

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

- GPU 있으면 "cuda:0", 없으면 "cpu" 로 설정됨.
- GPU 사용 방법
 - GPU 한 개일 때

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model = Net()
model.to(device)
```

- GPU 여러 개일 때

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Net()
if torch.cuda.device_count() > 1:
    model = nn.DataParallel(model) # 여러 GPU에 병렬 분산
model.to(device)
```

- nn.DataParallel(model) → 여러 GPU에서 병렬 처리.
- 주의: GPU 개수만큼 batch size를 늘려줘야 함.

Fashion-MNIST 데이터셋 불러오기

```
train_dataset = torchvision.datasets.FashionMNIST(
    "../chap05/data", download=True,
    transform=transforms.Compose([transforms.ToTensor()])
)

test_dataset = torchvision.datasets.FashionMNIST(
```

```

"../chap05/data", download=True, train=False,
transform=transforms.Compose([transforms.ToTensor()])
)

```

핵심 개념

- torchvision.datasets.FashionMNIST
 - PyTorch에서 제공하는 예제 데이터셋 (CIFAR, COCO, MNIST 등과 같은 하위 클래스 중 하나).
- 주요 파라미터:
 - 첫 번째 파라미터: 데이터를 저장할 위치 지정 (`"../chap05/data"`)
 - `download=True`: 지정된 경로에 데이터셋이 없으면 다운로드
 - `transform`: 데이터를 텐서(`0~1 범위`)로 변환

DataLoader에 데이터 전달

```

train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=100
)

test_loader = torch.utils.data.DataLoader(
    test_dataset, batch_size=100
)

```

핵심 개념

- DataLoader: 데이터셋을 미니배치 단위로 불러와 학습에 사용 가능하게 만드는 도구
- 주요 파라미터:
 - 첫 번째 파라미터: 어떤 데이터셋을 불러올지 지정 (`train_dataset` / `test_dataset`)
 - `batch_size`: 한 번에 가져올 데이터 개수 (여기서는 100개씩)
 - (추가) `shuffle=True`를 주면 데이터를 무작위 순서로 섞어 학습에 사용

클래스 정의

```
labels_map = {0:'T-Shirt', 1:'Trouser', 2:'Pullover', 3:'Dress',
              4:'Coat', 5:'Sandal', 6:'Shirt', 7:'Sneaker',
              8:'Bag', 9:'Ankle Boot'}
```

- 숫자 레이블(0~9)을 실제 클래스명(옷 종류)으로 매핑.

20개 무작위 이미지 출력



```
img_xy = np.random.randint(len(train_dataset))
img = train_dataset[img_xy][0][0,:,:]
```

- `np.random.randint(len(train_dataset))` → 전체 훈련 데이터 중 랜덤한 인덱스 선택.
- `train_dataset[img_xy]` → `(image_tensor, label)` 반환.
- `[0]` → 이미지 부분만 가져옴.
- `[0,:,:]` → 이미지 텐서에서 채널(1개, 흑백)만 선택해서 2D (28×28) 배열만 추출. 즉, 28×28 픽셀의 흑백 이미지 데이터를 가져옴.
- NumPy 랜덤 함수
 - `np.random.randint(10)` → 0~9 정수 중 임의 값.

- `np.random.rand(8)` → 0~1 사이 실수 8개 생성.
- `np.random.randn(4,2)` → 평균 0, 표준편차 1인 정규분포에서 난수 4×2 생성.
- NumPy 배열 인덱싱 예제

```

examp = np.arange(0, 100, 3) # 0~99까지 3씩 증가하는 배열
examp.resize(6,4)          # 6×4로 reshape
examp[3]                    # 4번째 행
examp[3][3]                 # 4번째 행, 4번째 열 값

```

- 이런 방식으로 `train_dataset[img_xy][0][0,:]` 같은 접근이 가능함.

심층 신경망 모델 (DNN)

모델 생성

- 모든 신경망 모델은 `torch.nn.Module` 을 상속받아 정의한다.
- `__init__()` : 모델이 처음 생성될 때 실행되는 초기화 함수. 여기서 레이어(층)를 정의한다.
- `forward()` : 데이터를 입력받아 레이어를 순서대로 통과시키는 함수. 실제 학습과 추론 시 반드시 실행된다.
- 코드 :

```

class FashionDNN(nn.Module):
    def __init__(self):
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256)
        self.drop = nn.Dropout(0.25)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 10)

    def forward(self, input_data):
        out = input_data.view(-1, 784) # 28×28 이미지를 784 벡터로 변환
        out = F.relu(self.fc1(out))    # 첫 번째 은닉층 + ReLU
        out = self.drop(out)           # Dropout 적용
        out = F.relu(self.fc2(out))    # 두 번째 은닉층 + ReLU
        out = self.fc3(out)            # 출력층 (10 클래스)
        return out

```

- `nn.Linear(in_features, out_features)`
 - 입력 크기(`in_features`)와 출력 크기(`out_features`)를 가지는 완전연결층(fully connected layer).
 - 예: `nn.Linear(784, 256)` → 784차원 입력 → 256차원 출력
- `nn.Dropout(p)`
 - 신경망 과적합을 방지하기 위해 입력 노드를 확률적으로 0으로 만든다.
 - `p=0.25` → 25% 확률로 입력을 0으로 만든다.
- `view(-1, 784)`
 - 입력 데이터를 reshape 하는 함수.
 - `28×28` 이미지를 1차원(784) 벡터로 변환한다.
 - `1`은 "자동 계산"을 의미 → 배치 크기에 따라 자동 조정된다.
- `F.relu()` VS `nn.ReLU()`
 - 두 방식 모두 ReLU 활성화 함수를 적용한다.
 - `F.relu()` → 함수 형태 (forward 안에서 사용)
 - `nn.ReLU()` → 모듈 형태 (init에서 정의 후 forward에서 호출)
 - 차이는 주로 호출 위치와 사용 방식에 있다.
- `nn.xx` 와 `nn.functional.xx` 비교

구분	<code>nn.xx</code>	<code>nn.functional.xx</code>
형태	<code>nn.Conv2d</code> : 클래스 <code>nn.Module</code> 클래스를 상속받아 사용	<code>nn.functional.conv2d</code> : 함수 def function (input)으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	<code>nn.Sequential</code> 내에 위치	<code>nn.Sequential</code> 에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

파라미터 정의

- `learning_rate = 0.001`
→ 학습 속도를 의미. 값이 크면 빠르지만 불안정, 작으면 안정적이지만 느림.
- `criterion = nn.CrossEntropyLoss()`

→ 분류 문제에서 자주 쓰이는 손실 함수 (예측값 vs 정답 비교).

- optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

→ 옵티마이저: 가중치 업데이트 방법. 여기서는 Adam 사용.

모델 학습 과정

- num_epochs = 5 → 데이터셋을 5번 반복 학습.
- 매 반복마다:
 - 데이터 불러오기: (images, labels)
 - 데이터 GPU로 이동: `.to(device)`
 - 데이터 모양 변환: `images.view(100, 1, 28, 28)`
→ 배치크기=100, 채널=1, 가로=28, 세로=28
 - 순전파 (Forward): `outputs = model(train)`
 - 손실 계산: `loss = criterion(outputs, labels)`
 - 역전파 (Backward):
 - `optimizer.zero_grad()` → 기울기 초기화
 - `loss.backward()` → 기울기 계산
 - `optimizer.step()` → 가중치 업데이트

검증

- 일정 간격마다(`count % 50 == 0`):
 - 테스트 데이터로 모델 평가
 - 예측값: `predictions = torch.max(outputs, 1)[1]`
→ 각 샘플마다 확률이 가장 높은 클래스 선택
 - 정확도 계산:
$$\text{accuracy} = \frac{\text{correct predictions}}{\text{total predictions}} \times 100$$
 - 오차율(error rate):
$$1 - \frac{\text{correct predictions}}{\text{total predictions}}$$

정확도 해석 시 주의점

- 80% 정확도
→ 모든 클래스가 고르게 잘 분류되었는지 알 수 없음 (데이터 불균형 주의).
- 90% 정확도
→ 예를 들어 100개 중 90개가 같은 클래스라면, 전부 그 클래스로 예측해도 90% 정확도가 나옴 → 데이터 분포 확인 필요.

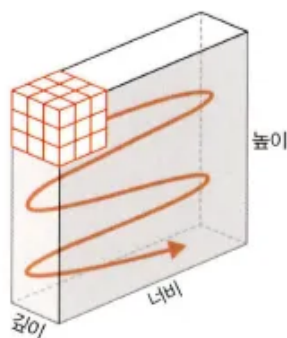
합성곱 네트워크 (CNN)

모델 생성

- `nn.Sequential`
 - 여러 개의 층(layer)을 차례대로 쌓아서 하나의 블록처럼 사용할 수 있게 함.
 - `__init__` 에서 정의 → `forward` 에서 순서대로 실행됨.
- 합성곱 계층 (Conv2d)

```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
```

- `in_channels`: 입력 채널 수 (흑백=1, RGB=3)
- `out_channels`: 출력 채널 수 (필터 개수, 즉 feature map 개수)
- `kernel_size`: 필터 크기 (예: 3 → 3x3 필터)
- `padding`: 입력 가장자리에 0을 채워서 출력 크기를 조절 (예: `padding=1` → 크기 유지)
- `stride`: 필터 이동 간격 (기본=1)



- 배치 정규화 (BatchNorm2d)

```
nn.BatchNorm2d(32)
```

- 미니배치 단위로 평균과 분산을 구해 정규화.
- 분포를 평균=0, 분산=1에 가깝게 만들어 학습 안정화 & 빠른 수렴.

- 활성화 함수 (ReLU)

```
nn.ReLU()
```

- 음수는 0, 양수는 그대로 통과.
- 비선형성 부여 → CNN이 복잡한 특징 학습 가능.

- 풀링 계층 (MaxPool2d)

```
nn.MaxPool2d(kernel_size=2, stride=2)
```

- 특징 맵의 크기를 줄여 계산량 감소, 불변성(translation invariance) 제공.
- Max Pooling: 영역 내 최대값 선택

- 완전연결층 (Fully Connected, Linear)

```
nn.Linear(in_features=64*6*6, out_features=600)
```

- 합성곱 + 풀링 후 결과를 1차원 벡터로 펼쳐(flatten) FC층에 입력.
- in_features: 입력 크기 (= Conv/Pool 결과 flatten한 값)
- out_features: 출력 크기 (= 뉴런 개수, 클래스 수)

- Dropout

```
nn.Dropout2d(0.25)
```

- 학습 시 랜덤하게 일부 뉴런을 끄 (0으로 만듦).
- 과적합 방지, 일반화 성능 향상.

- Forward 과정

```
def forward(self, x):  
    out = self.layer1(x)      # Conv1 + BN + ReLU + Pool
```

```

out = self.layer2(out)    # Conv2 + BN + ReLU + Pool
out = out.view(out.size(0), -1) # Flatten
out = self.fc1(out)       # Fully connected
out = self.drop(out)      # Dropout
out = self.fc2(out)       # Fully connected
out = self.fc3(out)       # Output layer (10 classes)
return out

```

- 예시: (FashionMNIST, 1×28×28 입력)

1. 입력: [1, 28, 28]
2. Conv1 (padding=1): [32, 28, 28]
3. Pool1 (2×2): [32, 14, 14]
4. Conv2 (kernel=3): [64, 12, 12]
5. Pool2 (2×2): [64, 6, 6]
6. Flatten: $64 * 6 * 6 = 2304$
7. FC1: $2304 \rightarrow 600$
8. Dropout
9. FC2: $600 \rightarrow 120$
10. FC3: $120 \rightarrow 10$

파라미터 정의

- 학습률(learning rate): 0.001
- 모델(model): `FashionCNN()`
- 손실 함수(loss function): `CrossEntropyLoss()`
- 옵티마이저(optimizer): `Adam` (모델 파라미터와 learning rate 사용)
→ 옵티마이저는 모델 가중치를 학습 방향으로 업데이트하는 알고리즘.

CNN 모델 구조

구성은 크게 합성곱 계층 + 완전연결층으로 나뉩니다.

1. 합성곱 계층
 - Conv2d: 이미지 특징 추출 (필터/커널 사용)

- BatchNorm2d: 배치 정규화, 학습 안정화 & 과적합 방지
- ReLU: 활성화 함수 → 비선형성 부여
- MaxPool2d: 이미지 크기 축소, 중요한 특징만 남김

2. 완전연결층

- Linear: Flatten된 특징 벡터를 입력받아 분류 수행
- Dropout: 일부 뉴런을 랜덤으로 꺼서 과적합 방지

학습 과정

반복(epoch)마다 다음 과정을 수행합니다.

1. Forward propagation

- 데이터를 모델에 넣어 출력(`outputs`) 계산

2. Loss 계산

- `CrossEntropyLoss(outputs, labels)`

3. Backward propagation

- `loss.backward()` 로 오차 역전파

4. Optimizer step

- `optimizer.step()` 으로 가중치 업데이트

5. 정확도 평가

- `torch.max(outputs, 1)[1]` → 가장 높은 확률의 클래스 선택
- 예측과 정답 비교하여 accuracy 계산

학습 결과

- Iter 500 → Accuracy 약 87.9%
- Iter 3000 → Accuracy 약 89.9%
- CNN은 DNN보다 정확도가 더 높음

DNN vs CNN 비교

- DNN (심층신경망)
 - 입력 이미지를 1차원으로 평탄화(flatten) → 특징 추출 한계 있음

- 단순한 데이터에는 충분하지만, 복잡한 이미지 데이터에는 성능이 떨어짐
- CNN (합성곱 신경망)
 - 합성곱/풀링 계층을 통해 이미지의 공간적 구조(패턴, 모양 등) 보존
 - 이미지 특징 추출 + 분류 모두 가능 → 더 높은 정확도
 - DNN보다 학습에 적합